

SoGoodUtilityGovernance.sol

SoGoodUtilityGovernance.sol

Summary of Changes:

Renamed contract to
SoGoodUtilityGovernance

Token name/symbol: "SoGood Utility Governance" / "SUG"

Updated roles:

TIP_ROLE → for tipping utility

DISTRIBUTOR_ROLE → for
Learn2Earn-style airdrops

All variables, events, comments, and
logic updated for brand + clarity

Lock duration fully retained

```
// SPDX-License-Identifier: MIT
```

```
pragma solidity ^0.8.20;
```

```
import "@openzeppelin/contracts
```

```
/token/ERC20/extensions
```

```
/ERC20Burnable.sol";
```

```
import "@openzeppelin/contracts
```

```
/access/AccessControl.sol";
```

```
/**
```

```
 * @title SoGoodUtilityGovernance
```

```
 (SUG)
```

```
 * @dev Governance, Learn2Earn,
```

```
 Staking, and Proof-of-Utility tipping
```

```
 token.
```

```
 */
```

```
contract SoGoodUtilityGovernance is
```

```
ERC20Burnable, AccessControl {
```

```
    bytes32 public constant TIP_ROLE =
```

```
    keccak256("TIP_ROLE");
```

```
    bytes32 public constant
```

```
    DISTRIBUTOR_ROLE =
```

```
    keccak256("DISTRIBUTOR_ROLE");
```

```
    mapping(address => uint256) public
```

```
    unlockTimestamp;
```

```
    uint256 public immutable
```

```
    lockDuration;
```

```
    event Tipped(address indexed from,
```

```
    address indexed to, uint256 amount,
```

```
    string reason);
```

```
    event Airdropped(address indexed
```

```
    user, uint256 amount);
```

```
    constructor(uint256 _lockDuration)
```

```
ERC20("SoGood Utility Governance",
```

```
"SUG") {
```

```
    _grantRole(DEFAULT_ADMIN_ROLE,
```

```
    msg.sender);
```

```
        lockDuration = _lockDuration; //
```

```
e.g. 60 days = 5184000 seconds
```

```
    }
```

```
    function airdrop(address
```

```
    user, uint256 amount) external
```

```
    onlyRole(DISTRIBUTOR_ROLE) {
```

```
        _mint(user, amount);
```

```
        unlockTimestamp[user] =
```

```
        block.timestamp + lockDuration;
```

```
        emit Airdropped(user, amount);
```

```
    }
```

```
    function tip(address to, uint256
```

```
    amount, string calldata reason) external
```

```
    onlyRole(TIP_ROLE) {
```

```
        _mint(to, amount);
```

```
        emit Tipped(msg.sender, to,
```

```
        amount, reason);
```

```
    }
```

```
    function transfer(address to, uint256
```

```
    amount) public override returns (bool) {
```

```
        require(block.timestamp >=
```

```
        unlockTimestamp[msg.sender],
```

```
        "Tokens are time-locked");
```

```
        return super.transfer(to, amount);
```

```
    }
```

```
    function transferFrom(address from,
```

```
    address to, uint256 amount) public
```

```
    override returns (bool) {
```

```
        require(block.timestamp >=
```

```
        unlockTimestamp[from], "Tokens are
```

```
        time-locked");
```

```
        return super.transferFrom(from, to,
```

```
        amount);
```

```
    }
```

```
    function isUnlocked(address user)
```

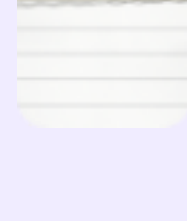
```
external view returns (bool) {
```

```
    return block.timestamp >=
```

```
    unlockTimestamp[user];
```

```
    }
```

```
}
```



Notein

Scan the QR code to download Notein

